

低密度奇偶校验码

peterrolls@qq.com

2026 年 5 月 30 日

低密度奇偶校验（LDPC）码是基于**校验矩阵**定义和构造的一类**线性分组码**，其校验矩阵为稀疏矩阵——大部分元素均为零，只有极少量的非零元素。在接收端，使用“和-积算法”（又称置信度传播算法）对该编码进行译码。

1 编码

主要讨论的是**二进制规则LDPC编码**。

一个二进制规则的LDPC码可以用以下参数定义：

编码长度	校验矩阵每列的“1”数量	校验矩阵每行的“1”数量
n	$p(p \geq 3)$	q

以上参数定义了一个 (n, p, q) 二进制规则LDPC码，记其校验矩阵为 $H_{m \times n}$ 。显然，由于“1”的数量确定， $m \cdot p = n \cdot q$ 。

如果校验矩阵 H 是满秩的，则编码码率 $R = (n - m)/n = 1 - q/p$ 。如果 H 的秩是 p' ，则该编码的码率为 $R = (n - p')/n$ 。

如果我们已经构建好了一个稀疏二进制 $m \times n$ 矩阵 H_a ，采用高斯消元法可以得到系统码形式的校验矩阵 $H_b = [I_m | -P]$ ，从而得到生成矩阵 $G \equiv [P | I_k]$ 。使用 G 矩阵就可以实现LDPC编码。但一般说来，生成矩阵 G 不是稀疏矩阵，其实现复杂度较高($O(n^2)$)。

记矩阵 H 为矩阵 H_a 在高斯消元执行标准化过程中，每进行一次列置换，对应进行相应列置换得到的结果。定义 H 为所构建的LDPC码的校验矩阵。容易知道，矩阵 H 与原矩阵 H_a 的稀疏程度完全相同，其所定义的编码仅仅是码元位置具有差异，因而也具有完全一致的Hamming距离特性。考虑到 H 是稀疏阵，使用 H 进行编码可以降低编码复杂度。

2 构建稀疏二进制矩阵 H_a

编码构造的任务就是构造一个二进制 $m \times n$ 满秩稀疏矩阵 H_a ，使矩阵每行恰含 q 个“1”，每列恰含 $p = m \cdot q/n$ 个“1”，此处 $m = (1 - R) \cdot n$ 。

2.1 闭环

为了得到好的译码性能，构造 H_a 矩阵时还应尽量使矩阵中的最小“闭环”长度最大，亦或使长度小的闭环数量尽量少。

H_a 矩阵中的“闭环”是由水平线段和竖直线段连接起来的闭合路径，路径中每一顶点（即水平线段和竖直线段的交点）处均为非零元素。

“闭环”的“长度”是指构成该闭合路径的水平和竖直线段的“数量”。举例说明，下图是一个长度为6的“闭环”示例：

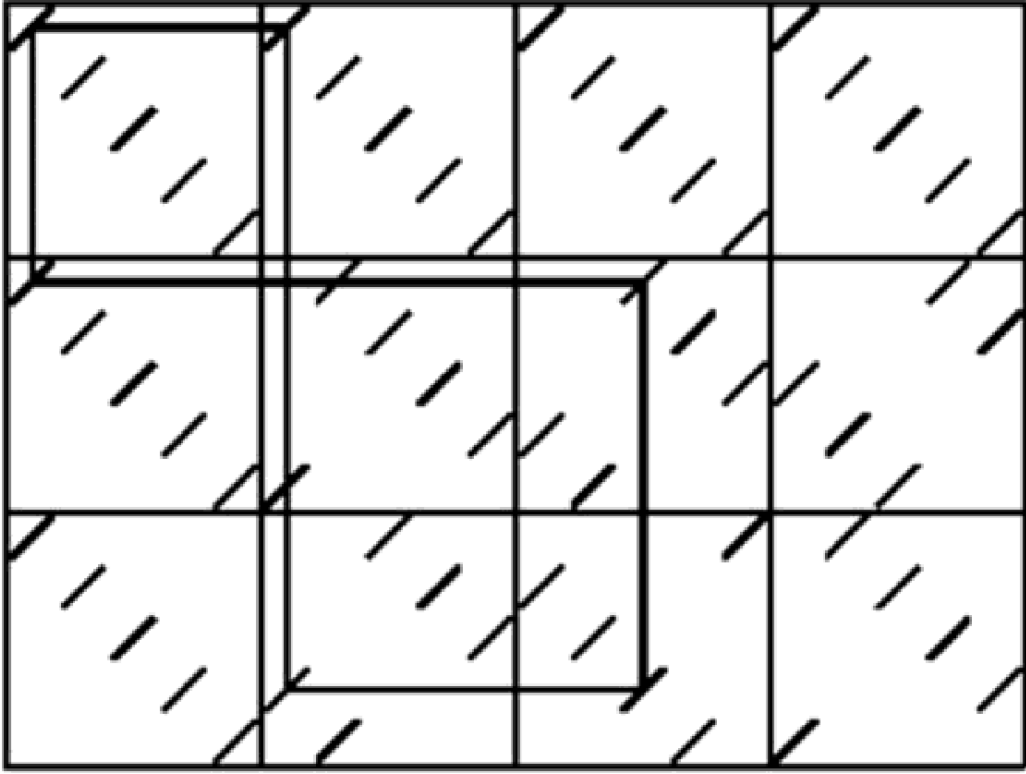


图 1: 校验矩阵中一个长度为6“闭环”的实例(空白表示“0”,斜线表示“1”)

2.2 Gallager方法

设置:

1. 每一行有 q 个“1”(满足行重要求);
2. 每一列有 p 个“1”(满足列重需求);
3. 任意两列具有共同“1”的个数不大于1(满足围长大于4的要求);
4. q 与 p 分别于 H_a 矩阵中的列数和行数相比小得多(满足稀疏性要求)。

设矩阵 H_0 为:

$$H_0 = \begin{bmatrix} \underbrace{11 \cdots 1}_q & & & \\ & \underbrace{11 \cdots 1}_q & & \\ & & \ddots & \\ & & & \underbrace{11 \cdots 1}_q \end{bmatrix}$$

利用 H_0 , 可以通过置换的方法得到校验阵 H_a :

$$H_a = \begin{bmatrix} \pi_1(H_0) \\ \pi_2(H_0) \\ \vdots \\ \pi_p(H_0) \end{bmatrix}$$

例如下图就是这种方法构造出的校验矩阵:

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ \hline 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ \hline 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

2.3 Mackay方法

此方法中，校验矩阵的列重为 j ，每行的平均重量为 k 。任意两行具有共同1的个数不大于1（保证闭环围长大于4）。

2.3.1 构造法1A

每一列具有固定的列重 j 。随机构造矩阵，使其平均行重为 k ，且任意两列具有共同1的个数不大于1。构造矩阵如图所示：

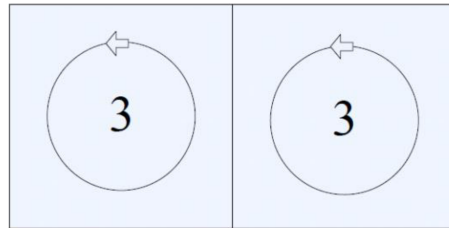


图 2: ($j = 3, k = 6, R = 0.5$)

可以按照上面方法生成一个 H 矩阵，例如：

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

2.3.2 构造法2A

与构造法1A类似，引入一些列重为2的列，使得 H 矩阵的围长增大，其中列重为2的列数为 $m/2$ ，这一部分是由两个 $m/2 \times m/2$ 的单位阵上下叠加起来构成的，如下图所示：

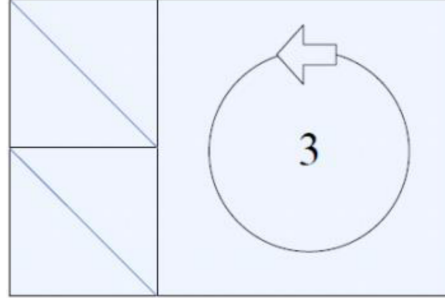


图 3: ($j = 3, k = 6, R = 0.5$)

可以按照上面方法生成一个 H 矩阵，例如：

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

2.3.3 构造法1B和2B

从构造法1A和2A中删除一些仔细选择的列，使得 H 的围长值满足相应的要求。

3 和-积译码算法

和-积译码可以用树图和因子图解释和理解。以Gallager方法构造的校验矩阵为例：

$$H = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ \hline 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ \hline 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

3.1 树图

基于Gallager方法构造的LDPC码奇偶校验矩阵，绘制出树图如4所示。

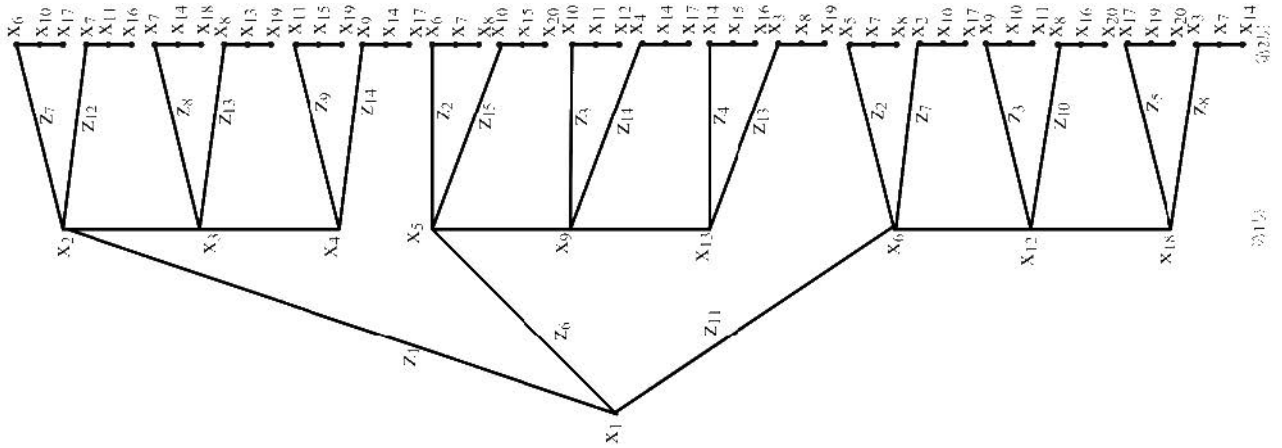


图 4: 校验码的树图表示(以(20, 3, 4)规则码为例, x_1 为起点)

3.1.1 怎么画的

记校验矩阵 H 的第 $1, 2, \dots, n$ 列分别与符号 $x_1, x_2, \dots, x_n$ 相对应, 它的第 $1, 2, \dots, m$ 行分别与校验方程 $z_1, z_2, \dots, z_m$ 相对应。

1. 取 x_1 为起点, 从校验矩阵 H 的第1列知, 校验方程 z_1, z_6, z_{11} 与 x_1 关联。以 x_1 为起点, 画出 z_1, z_6, z_{11} 树枝;
2. 观察校验矩阵 H 的第一行(对应 z_1), 发现除了 x_1 外, 还有 x_2, x_3, x_4 与 z_1 关联, 因此在枝 z_1 上标记 x_2, x_3, x_4 , 作为次级枝的起始节点;
3. 观察到 x_2 除了与 z_1 关联外, 还与 z_7, z_{12} 关联, 将 z_7, z_{12} 作为节点 x_2 的下一级枝画在图中;
4. 以此类推, 可以画出其他枝及其子节点, 其余子节点及其子枝, 从而得到树图。

3.1.2 怎么用的

译码从树图的高层逐层向下进行, 直到树根起始处作出该符号的判决为止。

假设第2层各符号的软信息(似然比)已得到, 则第1层各符号的软信息由两部分构成:

1. 信道传输后在接收端收到的本符号的直接信息;
2. 上层符号在校验方程约束下为本符号提供的外信息。

例如, 考虑第2层和第1层之间的枝 z_7 和 z_{12} :

$$z_7 : x_2 + x_6 + x_{10} + x_{17} = 0$$

$$z_{12} : x_2 + x_7 + x_{11} + x_{16} = 0$$

记 y 为对应 x_2 位置的直接接收符号并令 $s = \{z_7, z_{12}\}$ 。设发送符号先验概率未知（按等概计算），则：

$$\begin{aligned}
 \ln \frac{P(x_2 = 0|y, s)}{P(x_2 = 1|y, s)} &= \ln \frac{P(x_2 = 0|y, s)P(y, s)}{P(x_2 = 1|y, s)P(y, s)} = \ln \frac{P(x_2 = 0, y, s)}{P(x_2 = 1, y, s)} \\
 &= \ln \frac{P(y, s|x_2 = 0)P(x_2 = 0)}{P(y, s|x_2 = 1)P(x_2 = 1)} \\
 &= \ln \frac{P(y|x_2 = 0, s)P(s|x_2 = 0)P(x_2 = 0)}{P(y|x_2 = 1, s)P(s|x_2 = 1)P(x_2 = 1)} \\
 &= \underbrace{\ln \frac{P(y|x_2 = 0)}{P(y|x_2 = 1)}}_{\text{软信息}} + \underbrace{\ln \frac{P(x_2 = 0|s)}{P(x_2 = 1|s)}}_{\text{外信息}}
 \end{aligned}$$

当 x_2 确定时， y 是 x_2 的调制符号加上高斯噪声的结果，与 s 无关，是直接接收符号提供的“软信息”；校验方程的约束下其他符号为符号 x_2 提供的“外信息”。

注意到 z_7 和 z_{12} 是两个独立的校验方程，从而有：

$$P(s|x_2 = a) = P(z_7|x_2 = a)P(z_{12}|x_2 = a)$$

将此代入到上面似然比的推导中，得到：

$$\ln \frac{P(x_2 = 0|y, s)}{P(x_2 = 1|y, s)} = \ln \frac{P(y|x_2 = 0)}{P(y|x_2 = 1)} + \ln \frac{P(x_2 = 0|z_7)}{P(x_2 = 1|z_7)} + \ln \frac{P(x_2 = 0|z_{12})}{P(x_2 = 1|z_{12})}$$

以上的推导说明，在已知第 s 层所有符号的概率密度的情况下，根据第 $s-1$ 层符号节点与第 s 层树图之间的树枝，可以获得本层符号的似然比较信息。逐层向下操作直到树图底部即可译出全部软信息。

该译码过程说明，树图的独立层数越多，译码性能应该越好。构建校验矩阵时可采取适当措施尽量增大最小闭环长度，同时使长度小的闭环数量尽量小。

3.2 因子图

因子图（factor graph）也叫双向图（bipartite graph），将符号节点 $x_1, x_2, \dots, x_n$ 排成一行，同时将节点 $z_1, z_2, \dots, z_m$ 排成一行。如果校验方程第 i 行第 j 列的对应元素不为0，则称节点 x_j 和节点 z_i 之间关联，并用线段将两节点连接起来，如图5所示。

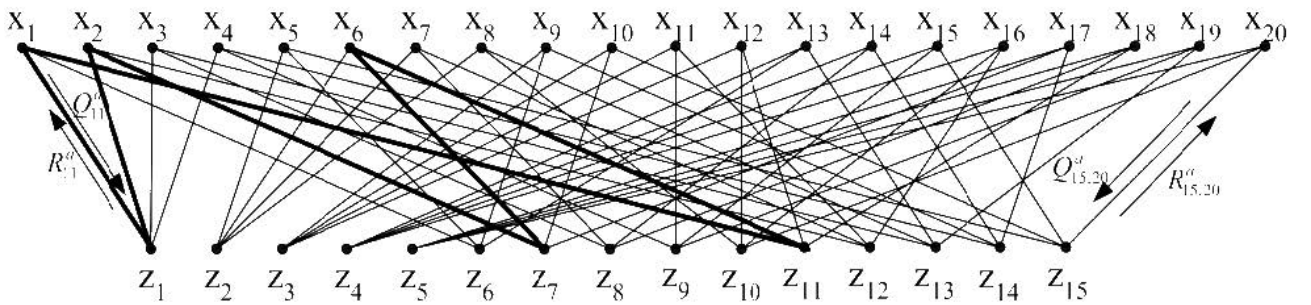


图 5: 校验码的因子图表示(以(20, 3, 4)规则码为例，图中粗黑线表示闭环)

采用因子图进行迭代译码过程中的信息传递，等效于树图的层之间的信息传递，只是表示方法略有差异而已。

具体在某一轮迭代中，符号节点和校验节点之间有信息传递，这种信息称为置信度。置信度实际是节点的某种条件概率。符号节点向校验节点传递的信息记为 Q ，而校验节点向符号节点传递的信息记为 R 。

3.3 算法流程

首先规定一些符号：

记 $f_j^a = P(x_j = a)$ 表示符号 x_j 的先验概率密度。该先验概率密度可以通过信道得到的直接信息和接收端对发送信号的先验知识等推导出来。例如对于采用BSC信道传输的二进制LDPC码, f_j^1 和 f_j^0 分别与互转移概率 ($P(0/1)$ 和 $P(1/0)$) 和自转移概率 ($P(0/0)$ 和 $P(1/1)$) 对应。

记 R_{ij}^a 表示在 $x_j = a$ 的条件下满足校验方程 z_i 的概率。该方程与 x_j 关联, 即因子图中 x_j 与 z_i 相连接。

记 $\mathfrak{N}(i) = \{s(i, 1), s(i, 2), \dots, s(i, h_i)\}$ 为与校验节点 z_i 相关联的所有符号节点的序号构成的集合。其中 h_i 表示该集合元素的个数。

校验方程 z_i 写为:

$$H_{i,s(i,1)}x_{s(i,1)} + H_{i,s(i,2)}x_{s(i,2)} + \dots + H_{i,s(i,h_i)}x_{s(i,h_i)} = 0$$

对于二进制LDPC码, $H_{i,u} = 1, u = s(i, 1), s(i, 2), \dots, s(i, h_i)$ 。

记 $M(j) = \{t(j, 1), t(j, 2), \dots, t(j, l_j)\}$ 为与符号节点 x_j 关联的校验符号节点的序号构成的集合。其中 l_j 表示该集合元素的个数。如果 z_i 与 x_j 相关联, 则 $i \in M(j)$ 。

记 Q_{ij}^a 表示除了节点 z_i , 所有与 x_j 关联的其他校验节点所对应校验方程组约束条件下, 节点 x_j 处于状态 a 的概率。即 $Q_{ij}^a = P(x_j = a | \{z_s\}_{s \in M(j) \setminus i})$, 其中 $M(j) \setminus i$ 表示除了 i 之外 $M(j)$ 集合中所有序号。

采用如下过程进行LDPC码的译码:

3.3.1 初始化

置 $Q_{ij}^a = f_j^a, i = 1, 2, \dots, m, j = 1, 2, \dots, n$

3.3.2 更新 R_{ij}^a

设与 z_i 关联的所有符号节点为 $x_{s(i,1)}, x_{s(i,2)}, \dots, x_{s(i,h_i)}$, 取其中的一个符号 x_j , 由于:

$$\begin{aligned} R_{ij}^a &= P(z_i | x_j = a) \\ &= \sum_{x_{s(i,1)}, x_{s(i,2)}, \dots, x_{s(i,h_i)} \setminus x_j} P(H_{i,s(i,1)}x_{s(i,1)} + H_{i,s(i,2)}x_{s(i,2)} + \dots + H_{i,s(i,h_i)}x_{s(i,h_i)} = 0 | x_j = a) \\ &= \sum_{x_{s(i,1)}, x_{s(i,2)}, \dots, x_{s(i,h_i)} \setminus x_j} \prod_{k \in \mathfrak{N}(i) \setminus j} Q_{ik}^{x_k} \\ &\quad z_{ij} x_j = a \end{aligned}$$

其中的加法次数随着 h_i 的增加而指数上升, 实际应用中, 一般采用“前向-反向迭代算法”或者FHT算法。FHT算法复杂度低, 适用于基于有限域 $GF(2^m)$ 的LDPC码。“前向-反向迭代算法”适用于一般LDPC码。

3.3.3 更新 Q_{ij}^a

设与 x_j 关联的所有校验节点为 $z_{t(i_1)}, z_{t(i_2)}, \dots, z_{t(i_{l_j})}$, 取其中的一个校验节点 z_i , 如果各校验方程独立, 则:

$$\begin{aligned} Q_{ij}^a &= P(x_j = a | \{z_s\}_{s \in M(j) \setminus i}) \\ &= P(x_j = a | \{z_{t(i_1)}, z_{t(i_2)}, \dots, z_{t(i_{l_j})} \setminus z_i\}) \\ &= \alpha_{i,j} f_j^a \prod_{k \in \mathfrak{N}(i) \setminus j} R_{kj}^a \end{aligned}$$

其中 $\alpha_{i,j}$ 为使 $\sum_a Q_{ij}^a = 1$ 的归一化因子。这样就得到了已知 R 信息更新 Q 信息的一般形式。

3.3.4 假设译码

对 $x_1, x_2, \dots, x_n$ 进行假设译码。已知有 $z_1, z_2, \dots, z_m$ 共 m 个校验约束，注意到每个符号 x_j 只与校验方程 $z_{t(i,1)}, z_{t(i,2)}, \dots, z_{t(i,l_j)}$ 有关，故：

$$\begin{aligned}
 & P(x_j = a, \{z_{t(i,1)}, z_{t(i,2)}, \dots, z_{t(i,l_j)}\}) \\
 &= P(x_j = a)P(\{z_{t(i,1)}, z_{t(i,2)}, \dots, z_{t(i,l_j)} | x_j = a\}) \\
 &= P(x_j = a) \prod_{k \in M(j)} P(z_k | x_j = a) \\
 &= f_j^a \prod_{k \in M(j)} R_{kj}^a
 \end{aligned}$$

现取：

$$\hat{x}_j = \arg \max_a f_j^a \prod_{k \in M(j)} R_{kj}^a$$

作为符号 x_j 的假设译码结果。如果译出的结果 $\hat{x}_1, \hat{x}_2, \dots, \hat{x}_n$ 满足校验方程，则可停止计算输出结果。否则再次更新 R 信息和 Q 信息。